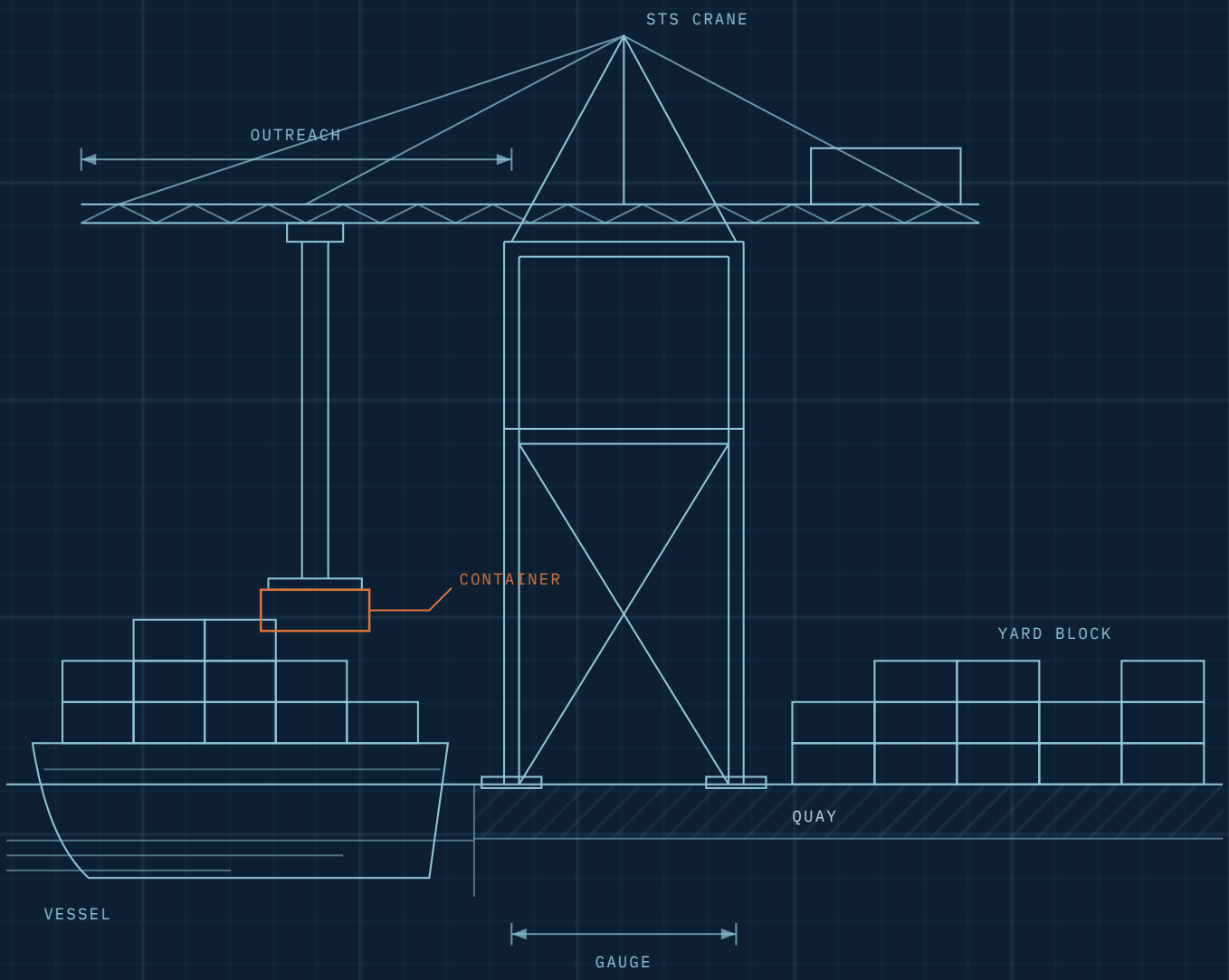
# Container Audit App: *System Plan*



# Contents


01 Context 03

1.1 Purpose and scope 04
1.2 The port ecosystem and where our app enters 05
1.3 Container journey and audit locations 07

02 Integration and architecture 09

2.1 KWATOS integration 10
2.2 System architecture 11
2.3 End-to-end audit flow 13

03 Computer vision pipeline 14

3.1 Container number OCR (on device) 15
3.2 ISO 6346 check digit 16
3.3 Colour detection (server CV worker) 17
3.4 Second-pass OCR (server) 17

04 Data model (Prisma) 18

4.1 Snapshots, scans and container events 19
4.2 Schema 20

05 API, real time and sync 23

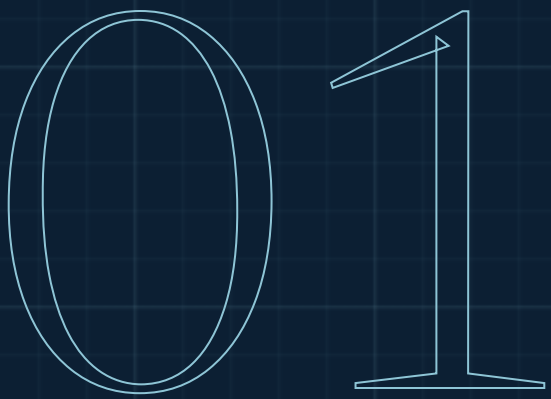
5.1 REST routes (Fastify) 24
5.2 Socket.IO events 25
5.3 Offline-first sync 26

06 Hosting and costs 27

6.1 Hosting on Contabo VPS 28
6.2 How it is deployed 29
6.3 Subscriptions and AI 30
6.4 Other subscriptions 31

07 Delivery 32

7.1 Build phases 33
7.2 Risks and open questions 34



# Context


1.1 Purpose and scope 04

1.2 The port ecosystem and where our app enters 05

1.3 Container journey and audit locations 07

# Purpose and scope

The app lets a yard crew prove, container by container, that what KWATOS says is in the yard is physically there. A supervisor imports the KWATOS Excel export as the expected list; crew members scan each box with a phone camera; the app reads the container number, checks it, records a photo and the box colour, and marks it found, missing or unexpected in real time.

| IN SCOPE FOR VERSION 1 | 07 ITEMS |
|---|---|
| V1.1 | Import a KWATOS container list (`.xlsx / .xls / .csv`) with a saved column mapping |
| V1.2 | Scan the container number from the door end with on-device OCR, validated by the ISO 6346 check digit |
| V1.3 | Capture a photo as evidence, with time, GPS and auditor name |
| V1.4 | Detect the dominant body colour and let the crew confirm it with one tap |
| V1.5 | Live progress on a supervisor dashboard while several crew members audit the same yard |
| V1.6 | Work offline in the yard and sync when signal returns |
| V1.7 | Export an audit report: found, missing, unexpected, mismatched, with photos |

**OUT OF SCOPE FOR VERSION 1**

Writing anything back into KWATOS, damage classification by AI, and seal-number verification.

# The port ecosystem and where our app enters

Our app sits downstream of KWATOS: it takes the container list KWATOS already holds, and adds the one thing KWATOS cannot do on its own, physical proof from the yard.

A terminal operating system (TOS) is the software a port runs its container operations on: where each box sits in the yard, its status, the vessel it belongs to, customs holds, moves and billing. KWATOS (Kilindini Waterfront Automated Terminal Operating System) is KPA's TOS. It went live in July 2008 at version 7.1, later moved to a web-based platform, and is integrated with the Nairobi and Kisumu ICDs, the National Single Window, KRA's Simba system and the Single Customs Territory (TradeMark Africa, KPA).

| TOS | LIVE SINCE | FIRST VERSION |
|---|---|---|
| KWATOS | July 2008 | 7.1 |

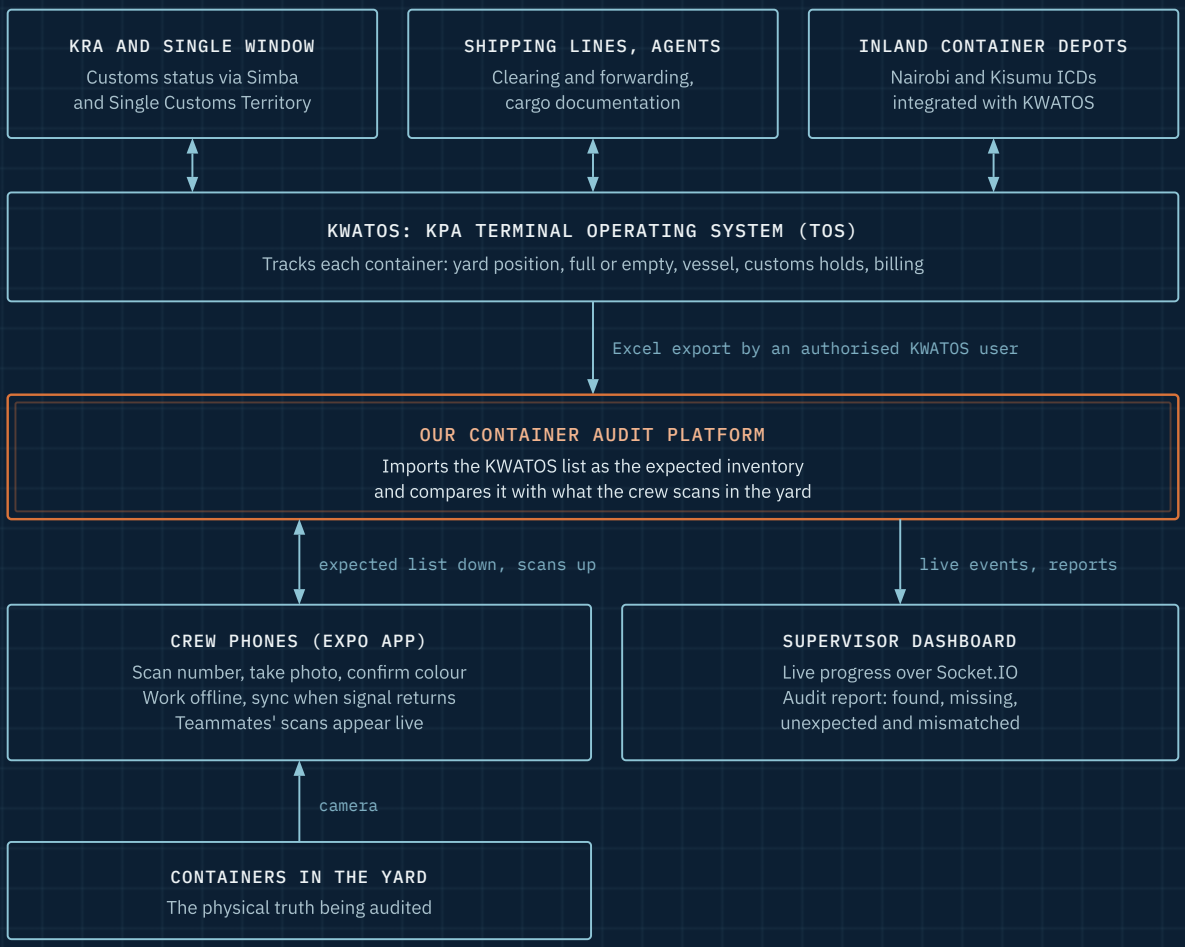
KEY POINT

The only bridge between KWATOS and our platform is a file exported by someone with KWATOS access. Everything after that (scanning, matching, photos, colour, reports) happens in our own system.

FIG. 01

## Our app reads KWATOS exports and checks them against the yard

Data leaves KWATOS as a file; nothing is written back into it



# Container journey and audit locations

A container does not stay in one place: after discharge it is routed by final destination, so the app audits one section at a time and links every scan into each box's journey.

FIG. 02 Containers move through several sections, and each one is a place to audit

Import flow at Mombasa under the KPA and KRA routing rules

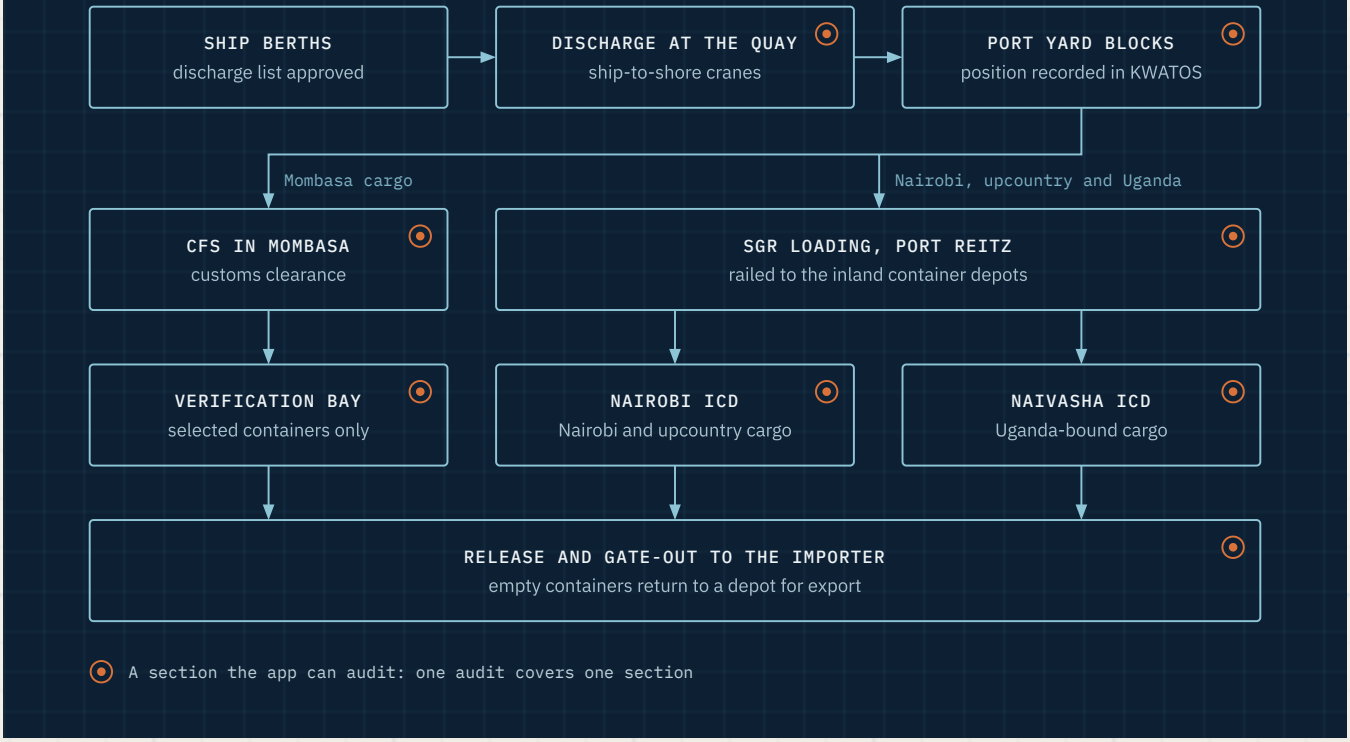



FIG. 02 · CONTAINER JOURNEY AT MOMBASA · AUDIT POINTS

Under the KPA and KRA arrangement announced in November 2025, Mombasa-bound containers are cleared at Container Freight Stations, Nairobi and upcountry cargo is railed to the Nairobi ICD, and Uganda-bound cargo goes to the Naivasha ICD; importers can nominate a CFS, otherwise KPA allocates one (Dawan Africa, The Kenya Times). Government agency interventions now happen outside the port and must be resolved within five days.

## What this means for the app

| | |
|---|---|
| FACILITIES AND SECTIONS | Each place in the diagram is a facility (port, a named CFS, an ICD) with sections (yard block, verification bay, SGR loading, gate). An audit covers one section, and its expected list is the KWATOS rows whose location codes map to that section. |
| CONTAINER EVENTS | Every scan and every import writes an event (discharged, in yard, moved to CFS, railed, verified, released, gate-out, audited), so the client can see where a box was last confirmed, by whom, with a photo. |
| SCOPE FOR VERSION 1 | Start with the sections the client's crew actually works in; the rest are configuration, not new code. |

Q1

OPEN QUESTION FOR THE CLIENT

Which of these sections does their crew audit today?



# Integration and architecture


2.1 KWATOS integration 10

2.2 System architecture 11

2.3 End-to-end audit flow 13

# KWATOS integration

KWATOS has no public API that we could find, so version 1 reads Excel exports, and the import is built as a swappable adapter so a better feed can replace it later without touching the rest of the app.

KPA shares data with the port community mainly through EDI messaging and system links to KRA and the Single Window; KPA's tender for a new TOS lists EDI messaging to the port community among the existing systems to integrate (Business Daily). Direct access to KWATOS would need a formal agreement with KPA ICT.

Every adapter outputs the same thing: a list of expected containers per location, plus movement events. The audit, scanning and reporting code never knows which source the data came from.

TABLE 02.1 IMPORT ADAPTERS

| SOURCE | WHEN | HOW THE APP USES IT |
|---|---|---|
| **Excel export** (`.xlsx / .xls / .csv`) | VERSION 1 | Uploaded on the dashboard, mapped once, validated with ISO 6346 |
| **EDI messages** (container discharge, load, gate-in / gate-out reports) | If the client already receives them from KPA or the shipping lines | Parsed automatically into container events, no manual export |
| **A KPA API** | Only if KPA's new TOS offers one | A new adapter; nothing else changes |

Q2

OPEN QUESTION FOR THE CLIENT

Do they already receive EDI files from KPA or their shipping lines, and can we get a sample?

# System architecture

Everything goes through one Fastify API: it owns authentication, the database (through Prisma), real-time broadcasts (Socket.IO) and the hand-off of photos to the computer vision worker.

FIG. 03 One Fastify API sits between the phones and every backend service

Phones never talk to the database or the CV worker directly

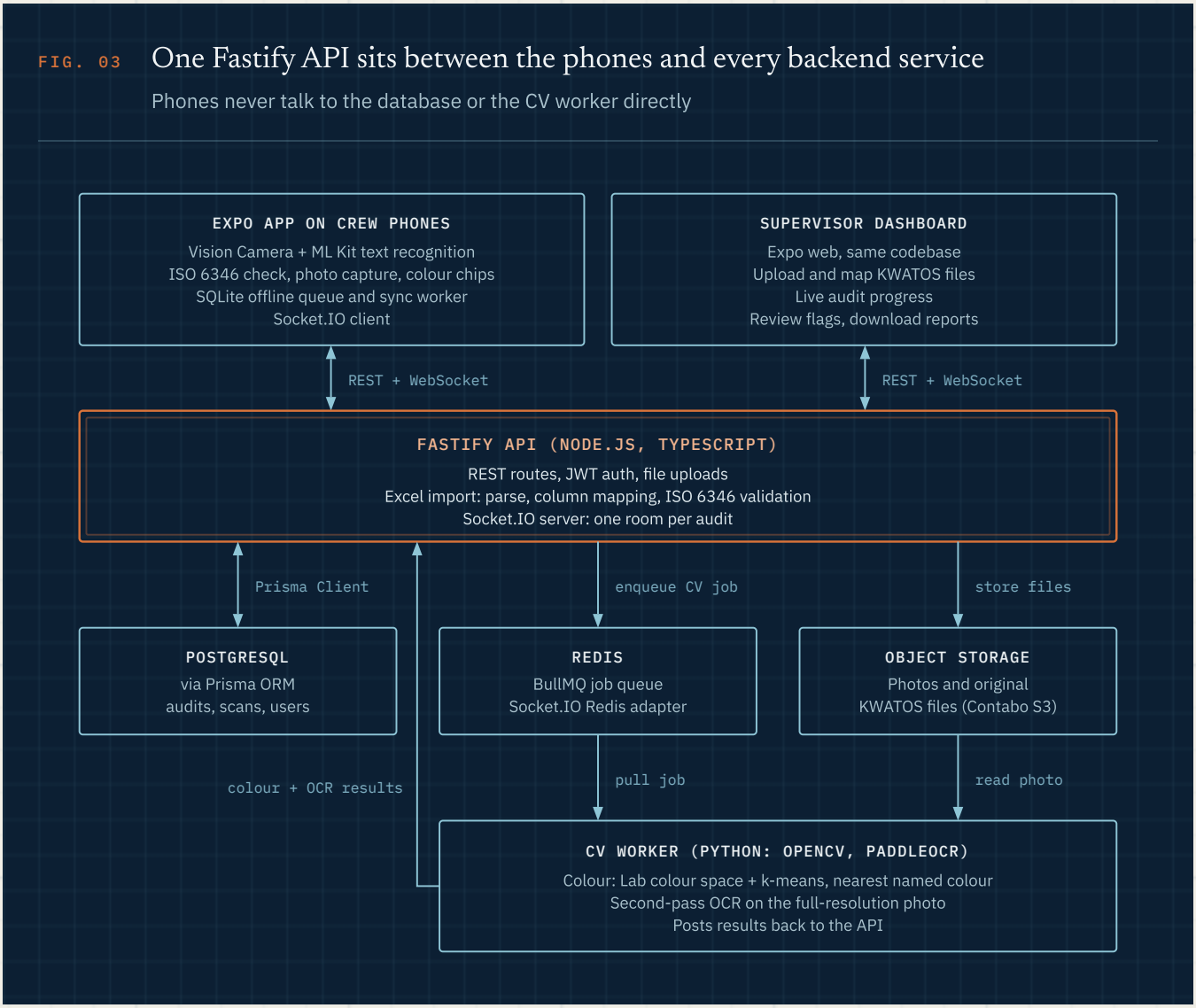



FIG. 03 · SYSTEM ARCHITECTURE · EXPO, FASTIFY, SOCKET.IO, PRISMA, CV WORKER

TABLE 02.2 PARTS AND TECHNOLOGY

| PART | TECHNOLOGY | WHY |
|---|---|---|
| **Mobile app** | Expo (React Native, TypeScript), dev build, `react-native-vision-camera`, ML Kit text recognition, `expo-sqlite` | One codebase for Android and iOS; on-device OCR works without signal |
| **Dashboard** | Expo web (Expo Router) | Reuses the app's components and API client |
| **API** | Fastify + TypeScript, Socket.IO, SheetJS or ExcelJS for imports | Fast, schema-validated routes; real time on the same server |
| **Database** | PostgreSQL + Prisma ORM | Typed queries and migrations; relational data fits audits well |
| **Queue** | Redis + BullMQ | Photos are processed in the background; retries are built in |
| **Computer vision** | Python worker with OpenCV and PaddleOCR | The strongest CV and OCR libraries live in Python; Node with sharp can do colour alone if the team prefers one language |
| **Files** | Contabo Object Storage (S3-compatible) | Photos and original KWATOS files stay out of the database; all services run in Docker on two Contabo VPS servers (see Hosting) |

REPOSITORY LAYOUT

A pnpm monorepo with shared packages, so the app and the API validate data with the same code.

| | |
|---|---|
| `apps/mobile` | `apps/api` |
| `services/cv-worker` | `packages/schemas` |
| `packages/iso6346` | |

# End-to-end audit flow

An audit starts with a KWATOS export and ends with a report where every container is in one of four buckets: matched, unexpected, missing, or mismatched (number or colour disagrees with a second check).

FIG. 04 Every container ends up matched, unexpected or missing

One audit, from KWATOS export to report

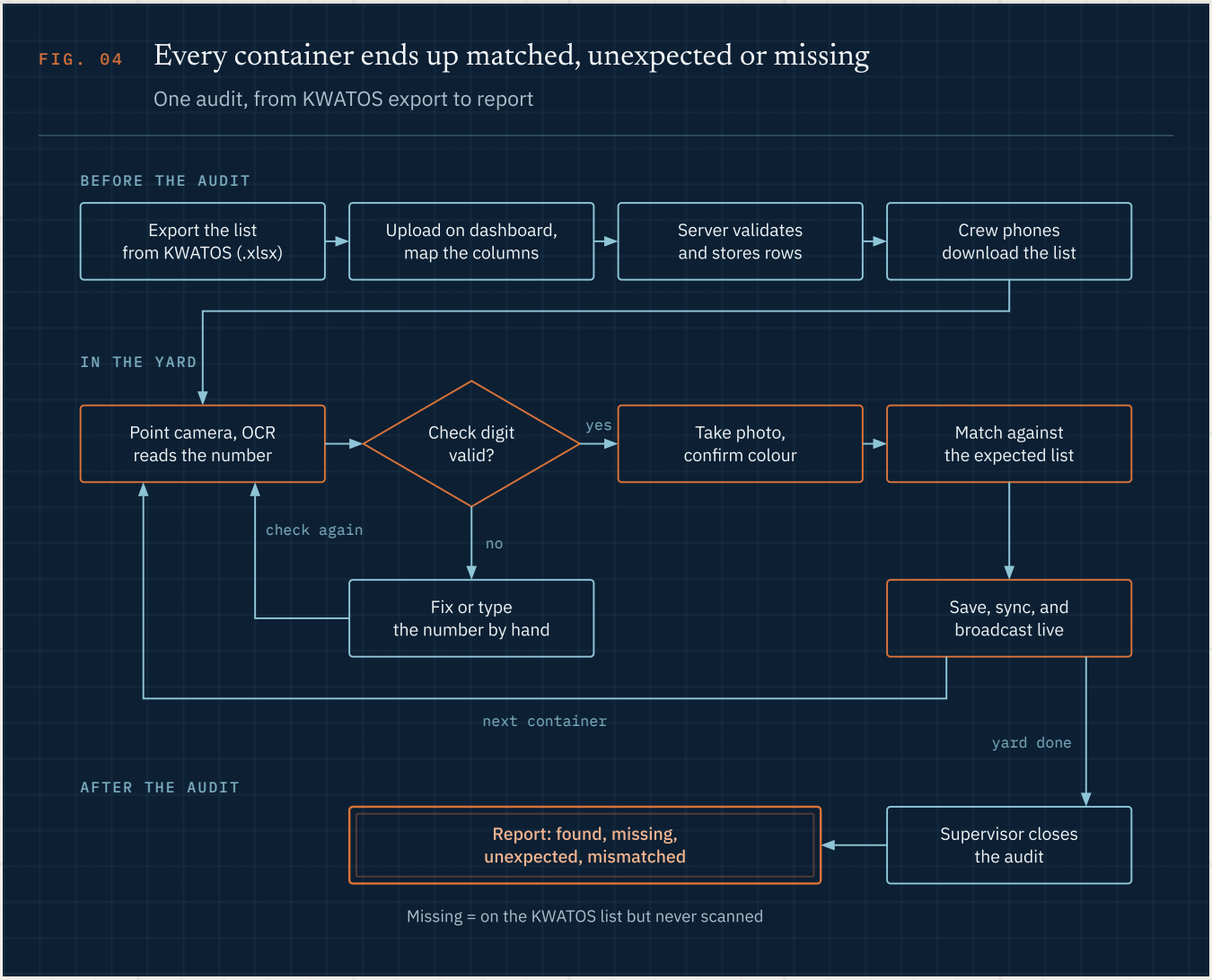



FIG. 04 · AUDIT FLOW · BEFORE, IN THE YARD, AFTER

The yard loop runs on the phone without signal: OCR, check digit, photo, colour and matching all happen against the local copy of the list. Only "broadcast live" needs a connection, and queued scans broadcast as soon as the phone reconnects.

# 03

# Computer vision pipeline


3.1 Container number OCR (on device) 15

3.2 ISO 6346 check digit 16

3.3 Colour detection (server CV worker) 17

3.4 Second-pass OCR (server) 17

Reading the number happens on the phone so it works with no signal; colour detection and a second OCR pass happen on the server once the photo uploads. The crew never waits on the network to keep moving.

## 01 Container number OCR (on device)

Expo Go cannot run native ML libraries, so the app ships as an Expo development build (`expo-dev-client`) with `react-native-vision-camera` and an ML Kit text-recognition frame processor. The camera shows a guide box over the top-right of the door, where the number is painted.

RAW OCR TEXT IS CLEANED BEFORE IT IS TRUSTED — 04 STEPS

01 Strip spaces and dashes; join the owner code and the serial if they were read as two lines.

02 Match the pattern of 4 letters + 6 digits + 1 check digit; the 4th letter must be `U`, `J` or `Z`.

03 Fix common confusions by position:

| IN THE LETTER PART | IN THE DIGIT PART |
|---|---|
| `0→O 1→I 5→S 8→B` | `O→0 I→1 S→5 B→8` |

04 Validate the check digit (below). **Pass** → auto-accept with a short vibration. **Fail** → show the read and ask the crew to correct it.

## 02 ISO 6346 check digit

Each letter maps to a number (A=10, then skipping multiples of 11: B=12 … K=21, L=23 … U=32, V=34 … Z=38); digits keep their value. Each of the first 10 characters is weighted by 2 to the power of its position:

SPEC 03.2 · CHECK DIGIT — PACKAGES/ISO6346

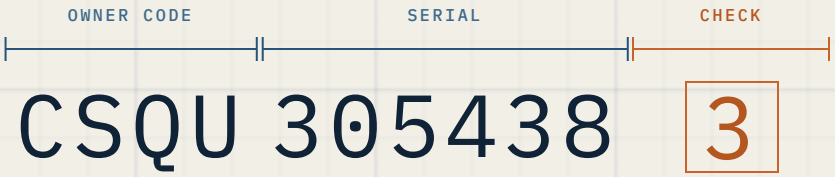
$$check = \left( \sum_{i=0}^{9} v_i \cdot 2^i \bmod 11 \right) \bmod 10$$

```
\text{check} = \left( \sum_{i=0}^{9} v_i \cdot 2^{i} \bmod 11 \right) \bmod 10
```

LETTER VALUES

| A | B | … | K | L | … | U | V | … | Z | skipping multiples of 11; digits keep their value |
|---|---|---|---|---|---|---|---|---|---|---|
| 10 | 12 | … | 21 | 23 | … | 32 | 34 | … | 38 | |

WORKED EXAMPLE




`CSQU305438` gives check digit 3, so **`CSQU3054383`** is valid. The same function lives in a shared `packages/iso6346` module used by the app, the API and the Excel importer, so a bad number in the KWATOS export is flagged too.

## 03 Colour detection (server CV worker)

01 Crop the container body (below the number guide box, away from sky and ground) and downscale it.

02 Convert pixels to the Lab colour space, which matches human colour judgement better than RGB.

03 Run k-means with a few clusters; drop near-black (shadow) and near-white (glare) clusters; keep the largest remaining one.

04 Match it to the nearest named colour in a fixed palette:

`blue` `red` `maroon` `green` `grey` `white` `orange` `yellow` `brown` `black`

05 Return the colour name, its hex and a confidence. **High confidence** → pre-selected chip; **low** → the crew picks.

Offline, the crew taps a colour chip directly; the server's suggestion is stored beside it when the photo syncs, so disagreements show up in the report.

## 04 Second-pass OCR (server)

The CV worker re-reads the number from the full-resolution photo. If it disagrees with the device read, the scan is flagged for supervisor review instead of silently changed.